

Kumkum Payal – Implementation Prompts

📅 8 Oct 2026 · @Snuggles

How to use these prompts

Run one Master Prompt plus twelve phase prompts, one phase at a time, so the AI audits the existing app before changing it.

The live app was built with v0, so it is most likely Next.js. Its header shows a **Demo Role Preview · Switch Role** toggle, and the dashboard date did not match the current date when checked. Both hint that roles may be only a front-end switch and some data may be mock or statically rendered. If so, staff isolation (Req 1) cannot work until real auth exists, which is why Phase 1 checks this first.

1. Export the v0 project to GitHub and open it in Claude Code (or Cursor).
2. Save the client requirements document in the repo as `docs/CLIENT_REQUIREMENTS.md`. The prompts refer to it by section number instead of repeating it.
3. Save the Master Prompt as `CLAUDE.md` in the repo root. Claude Code reads that file automatically every session, so you only paste the phase prompts.
4. Run each phase on its own git branch. Back up the database before any migration, and test on a Vercel preview before merging.
5. Settle the client questions at the end of this document before Phase 2, and paste each answer into the matching bracketed line in the prompts.

Master Prompt (save as CLAUDE.md)

This prompt sets the rules for every phase: read before writing, server-side security, atomic accounting saves, and honest reporting.

MASTER PROMPT – Kumkum Payal: implementing client changes

You are a senior full-stack engineer working on an EXISTING app: 'Kumkum Payal – Jewellery Accounts' (live: <https://lumina-jet-mu-30.vercel.app/>, originally generated with v0, deployed on Vercel). You will implement the client changes in `docs/CLIENT_REQUIREMENTS.md` (sections 1-52) by modifying the existing code. This is NOT a rebuild and NOT a visual redesign. These are business logic, accounting, permissions, automation, UX and data-integrity changes.

SOURCES OF TRUTH

- docs/CLIENT_REQUIREMENTS.md – what the client wants. Refer to requirements by number ('Req 33').
- docs/AUDIT.md + docs/CHANGE_MAP.md – what exists and what will change (created in Phase 1).
- docs/PROGRESS.md – running log. Update it at the end of every phase: done, files touched, migrations, open questions, next step. At the start of any new session, read it first.

NON-NEGOTIABLE RULES

1. Read before you write. Never modify a file you haven't read. Reuse existing components, tables, APIs, naming and styling.
2. Smallest change that fully satisfies the requirement. No drive-by refactors, renames, library swaps, or removal of working features.
3. Database changes only via new, additive, reversible migration files. Never drop/rename columns or tables that hold data. Backfill new columns safely.
4. Security is enforced on the server/database, never only in the UI. Never trust role, user id, timestamps or totals sent from the browser.
5. Timestamps come from the database clock (DEFAULT now()). The client never sends created_at.
6. Money never uses floating point. Use integer paise or a decimal type end to end. KG quantities support 3 decimals.
7. Every financial save is atomic: header + lines + ledger + stock + reminder + audit row + notification-outbox row in ONE database transaction. If any step fails, nothing is saved.
8. Balances are never typed by users. Balance Before / Closing Balance are always derived from the ledger using the app's existing debit/credit convention. Displayed balance must always equal the ledger balance.
9. Notifications/WhatsApp run only after commit, never block or roll back an accounting save, and never show 'sent' unless the provider confirmed delivery.
10. Every create/edit/delete writes an audit log row (user, action, entity, entity id, timestamp; before/after for edits).
11. Keyboard shortcuts must never interfere with normal typing.
12. Never claim something works unless you actually ran/tested it. State clearly what you could not test.

STOP AND ASK ME (don't guess) WHEN

- A requirement is ambiguous or conflicts with existing accounting logic.
 - A change would alter how existing balances or historical data compute.
 - You need credentials, an external service, or a destructive migration.
- Ask concise questions, each with your recommended option.

HOW TO WORK EACH PHASE

1. PLAN: files to touch, migrations, server/API changes, risks. Wait for my 'go'.
2. IMPLEMENT.

3. VERIFY: typecheck, lint, build, tests. Add automated tests for calculations, permissions and balances.
4. REPORT: files changed, migrations, API changes, how you tested, limitations. Update docs/PROGRESS.md.

Acknowledge these rules and wait for the phase prompt.

Phase 1 — Audit and change map (no code changes)

This phase produces AUDIT.md and CHANGE_MAP.md and changes no application code.

PHASE 1 – Audit the existing project and build the change map. DO NOT modify application code in this phase.

Create docs/AUDIT.md covering:

1. Stack: framework + version, router type, UI library, forms/state libraries, backend style (server actions / route handlers / external API), ORM/query layer, database + host, auth provider, env var NAMES used.
2. Is the data real? Confirm whether each page reads from a real database or from mock/seed/hardcoded/localStorage data. The live UI has a 'Demo Role Preview · Switch Role' toggle and the dashboard showed a date that wasn't today's. Determine whether these mean client-side role switching, mock data, or static rendering.
3. Auth & roles: how users log in, where the role is stored, whether the role is enforced anywhere server-side, and what 'Switch Role' actually does.
4. Data model: every table with columns, relations, constraints, indexes, RLS policies. Specifically: are customers/suppliers/karigars one table with a type, or separate? Do items have units/codes? Do entry tables have created_by?
5. Ledger: stored running balances or computed from transactions? Debit/credit sign convention for customer, supplier, karigar, cash, bank. Include one worked example traced through the code.
6. Each module – Entries (Sales, Purchase, Job Work, Receipt, Payment), Parties, Stock, Cash & Bank, Staff, Reminders, Audit Log, Dashboard: routes, components, data functions, what works, what's incomplete.
7. Existing: keyboard shortcut handling, bill numbering, bill view/print, any WhatsApp/notification code, any DB transaction usage (BEGIN/COMMIT or RPC), existing tests.
8. Risks and tech debt that affect these changes.

Then create docs/CHANGE_MAP.md: one row per requirement (Req 1-49) with columns:

Req		Exists today?		Required change		DB change		Server/API change		UI change		Permission change		Risk		Phase
-----	--	---------------	--	-----------------	--	-----------	--	-------------------	--	-----------	--	-------------------	--	------	--	-------

Finally, list 'Decisions needed from client': every ambiguity or conflict with the existing app, each with your recommended default.

Report a summary and stop.

Phase 2 — Real roles and staff isolation

This phase makes staff isolation real at the database/server level, before any feature work.

PHASE 2 — Real roles and staff isolation (Req 1, 2, 41; Req 51 Tests 1-3). Do this before any feature work.

Goal: Owner sees and manages everything. Staff can only retrieve their OWN entries, enforced at the database/server level.

1. Auth: if roles are currently a demo/client-side switch, implement real server-side role resolution from the logged-in user (users/profiles table, role 'owner' | 'staff'). Keep the demo switch only behind a dev-only env flag, never in production. Don't break existing login.
2. Ownership: every entry table (sales, purchases, job work, receipts, payments, orders, reminders; line tables via their parent) gets created_by (FK to user) and created_at DEFAULT now(). Migration backfills existing rows to the owner account. created_by is set on the server from the session; ignore any client value.
3. Enforcement:
 - If Postgres/Supabase: add RLS policies. Owner = full access; staff = SELECT/INSERT where created_by = current user; line tables inherit through the parent. Make sure no user-facing read uses a service-role/admin key.
 - Otherwise: one server-side scoping helper that EVERY entry query goes through — lists, get-by-id, search, exports, dashboard totals, bill/print views, reports.
 - Grep for every query touching these tables and list them in your report as 'scoped'.
 - Get-by-id for another staff member's record returns 404 (not 403) so its existence isn't revealed.
4. Edit/Delete: Owner only, checked on the server. Staff cannot edit/delete. [CHANGE THIS LINE if the client decides staff may edit their own entries.]
5. Party balances: staff may see a party's current balance figure on entry screens (needed for Req 13/23/31), but never the other staff members' underlying entries. [CHANGE if the client decides otherwise.]
6. Owner delete must reverse ALL effects in one transaction: ledger rows, stock movements, line items, linked reminders -> Cancelled. Write an audit row with a snapshot of the deleted record. No orphans. Prefer existing soft-delete if the app has one.

7. Owner edit recomputes ledger/stock effects atomically and audits before/after values.
8. Staff page: owner can create staff users and set roles.
9. Tests with Owner, Staff A, Staff B: Staff A creates a sale -> Staff B cannot see it via list, search, get-by-id, direct API call, or dashboard totals; Owner can. Staff edit/delete attempts are rejected by the server even when calling the API directly.

Phase 3 — Party, Supplier, Karigar masters + Alt shortcuts

This phase adds address and masked mobile to all three masters, plus Alt+C, Alt+S and Alt+K quick-add.

PHASE 3 — Customer/Supplier/Karigar masters and quick-add shortcuts (Req 8-10, 18-20, 27-29, 42).

1. Fields: Customer, Supplier and Karigar all support name, mobile, address (+ work type for Karigar, e.g. Polish, Meena, from a configurable list). Extend the existing party structure; if it's one table with a type, add columns there. Validate Indian mobile (10 digits, optional +91), store normalised.
2. Hide mobile after save (privacy, NOT deletion): keep the full number in the DB. In saved lists/details show it masked (e.g. ●●●●●4321). Owner gets a 'show' toggle; staff never receive the full number from the server (mask in the API response, not CSS). The full number remains available server-side for WhatsApp.
3. One reusable QuickAddPartyDialog(type) for all three types. On save: returns the new record, closes, auto-selects it in the form that opened it, moves focus to the next field. The bill's existing form data must be preserved.
4. Shortcuts via one shared hook (reuse existing if present):
 - Alt+C on Sales screens -> Add Customer
 - Alt+S on Purchase screens -> Add Supplier
 - Alt+K on Job Work screens -> Add KarigarMatch event.code (KeyC/KeyS/KeyK), not event.key, because on macOS Option+C types a special character. Call preventDefault (Firefox uses Alt+S for its History menu). Active only on the relevant screen; works even while focus is in an input; does nothing if a dialog is already open. Show the shortcut hint next to the party selector.
5. Audit log rows for party/supplier/karigar create/edit.
6. Test the shortcuts in Chrome, Edge and Firefox and report results.

Phase 4 — Shared fast-entry grid, save shortcut, Backspace

This phase builds one reusable item grid plus the save and back shortcuts that every bill and order form will share.

PHASE 4 — Shared fast-entry item grid, Save shortcut, Backspace-to-go-back (Req 11, 12, 15, 16, 21, 22, 30, 42, 43).

Build ONE reusable ItemEntryGrid (or adapt an existing line-item component) to be used by Sales, Purchase, Job Work, Sales Order and Purchase Order.

GRID

- Opens with one empty row with the item cell auto-focused, so the user can type immediately. No 'Add Item' click needed.
- Filling the item in the last row automatically adds a new empty row. Trailing empty rows are ignored and never sent to the server.
- Keyboard: Enter/Tab -> next cell; Enter on the row's last cell -> next row's item cell; Shift+Tab -> back; Ctrl+Delete removes the current row (always keep at least one empty row); arrow keys inside the item dropdown.
- Item cell: type-ahead by item name AND item short code. If items have no code, add an optional unique 'code' column.
- Units: each item's allowed units (PCS, KG, or both) come from item config. The unit cell offers only allowed units and defaults to the item's default unit. PCS = whole numbers; KG = up to 3 decimals (2.500).
- Rate is per selected unit (Rs/PCS or Rs/KG). Amount = qty x rate, live, rounded to paise, read-only.
- All maths in one pure, unit-tested function (no floats). Grid exposes rows + subtotal to the parent.
- Inline row validation: item selected, qty > 0, valid unit, rate >= 0.

SAVE SHORTCUT (shared hook)

- Ctrl+S / Cmd+S, plus Ctrl+Enter. preventDefault so the browser 'Save page' dialog never opens. Show the hint on the Save button.
- Same path as clicking Save: validate -> save -> success confirmation -> reset to one empty row, keep useful context (date, last selected bank account) -> focus the party field.
- No duplicate saves: disable during save (in-flight lock) AND send an idempotency key (UUID created when the form opens), stored server-side with a unique constraint, so double presses or retries can never create two bills.

BACKSPACE = BACK

- Only when the focused element is NOT editable (input, textarea, select, contenteditable, role=combobox/textbox/searchbox), no dialog/dropdown is open, and the user is on a child page/form. Never on the dashboard.

- If the form has unsaved data, confirm before leaving.

Add tests for the calculation function and the shortcut guard logic. Wiring into each module happens in the next phases.

Phase 5 — Sales Bill

This phase delivers the full Sales Bill: ledger-based old balance, extra charges, payment reminder, and an atomic save.

PHASE 5 – Sales Bill (Req 2, 11-17, 40, 46, 48, 49).

1. ItemEntryGrid for items; customer selector with Alt+C (Phase 3).
2. Old balance: on selecting a customer, fetch the current balance from the ledger (server) and show Previous Balance | This Bill | Closing Balance with Dr/Cr (or Receivable/Payable) labels per the existing sign convention. Closing balance updates live while the bill is edited; the server recomputes on save.
3. Charges below the grid: GST, Discount, Transport, Packaging, Other Charges.
 - GST rate is configurable (setting + per-bill override), never hard-coded. Discount as amount or %.
 - Follow the app's existing tax model. If none exists, propose an order (e.g. subtotal - discount = taxable value -> GST on taxable value -> + transport + packaging + other = Grand Total) and confirm it with me before finalising. [Paste client's answer here if you already have it.]
 - Store each component separately. One pure calculateBillTotals() shared by client and server, unit-tested. The server recalculates and rejects mismatched totals.
4. Optional 'Set payment reminder': date (required if enabled, not in the past), amount (default = bill total; state the choice), note. Saved in the same transaction, linked to sale + customer, using the existing Reminders module (extend its schema if needed).
5. Atomic save in one DB transaction: sale header (bill number from a race-safe sequence) -> lines -> ledger -> stock movements (if stock is tracked) -> reminder -> audit -> notification outbox row. If using Supabase, implement this as a Postgres function called via RPC, because separate supabase-js calls can't share a transaction.
6. Validation (client and server): customer required, >=1 valid line, qty > 0, unit valid for the item, rate >= 0, totals consistent.
7. UX: saving state, success toast with bill number, clear error messages, form reset (Phase 4), balances refresh immediately.
8. Saved bill view shows Balance Before | This Bill | Closing Balance computed from the ledger as at that entry (ordered by created_at, id), not stored copies that can drift after edits.

9. Owner edit/delete follows Phase 2 rules.

Tests: Req 51 Tests 4, 5, 8 (Rs 20,000 + Rs 5,000 -> Rs 25,000); decimal KG lines; double-save protection; full rollback when one step fails (simulate a failure).

Phase 6 — Purchase Bill

This phase mirrors Sales for purchases, reusing its components, and adds Narration.

PHASE 6 – Purchase Bill (Req 18-26). Mirror the Sales implementation and REUSE its components/functions; don't duplicate code.

1. Supplier selector + Alt+S quick add.
2. ItemEntryGrid with PCS and KG (e.g. 100 PCS x Rs 50; 2.500 KG x Rs X).
3. Supplier Previous Balance | This Purchase | Closing Balance from the ledger, with the correct supplier direction (a purchase increases what the business owes).
4. Extra charges if applicable, via the same calculateBillTotals().
5. Narration: free-text field (~500 chars), stored on the purchase, shown on view/print, searchable if the entries list has search.
6. Optional payment reminder linked to supplier + purchase.
7. Save shortcut, atomic save (incl. stock-in if tracked), audit, outbox – same pattern as Sales.
8. Validation: supplier required, >=1 line, valid qty/rate.

Tests: Req 51 Tests 6 and 13, plus a supplier balance before/after example.

Phase 7 — Job Work (Karigar)

This phase links each received-back line to what was actually sent, and shows the difference as negative (short) or positive (extra).

PHASE 7 – Job Work: send to Karigar, Received Back, Labour Charge (Req 27-33).

First understand how Job Work is modelled today and extend it. Target model (adapt to existing):

- Issue (sent): karigar, work type (Polish / Meena / configurable), lines: item, qty, unit, weight if KG, note.
- Receive (received back): karigar, lines each linked to a specific issue line, received qty/weight, labour charge.
- Pending per issue line = sent - sum(received against that line).

1. Karigar selector + Alt+K quick add. Show the Karigar's Previous Balance from the ledger.
2. Issue screen uses ItemEntryGrid (empty first row, fast entry).
3. Received Back: after choosing a karigar, list ONLY lines sent to THAT karigar with pending > 0, grouped by issue number/date. No free item selection here. Respect Phase 2 staff isolation.
4. Each line shows Sent | Received | Difference, where Difference = Received - Sent:
8 of 10 -> -2 (red); 12 of 10 -> +2 (green); 10 of 10 -> 0. For KG items show weight difference to 3 decimals.
5. Partial receipt: the remainder stays pending by default, with a 'Close line with shortage' option. Surplus is accepted and recorded.
6. Labour charge per line (fixed amount or rate x qty/weight; reuse existing logic if present). Posts to the Karigar ledger as payable when work is received back [confirm timing with me]. Show Previous Balance | Labour this entry | Closing Balance.
7. Stock (if tracked): items move to 'with karigar' on issue and back on receive, following existing stock logic, with no double counting.
8. Atomic saves, audit, outbox, save shortcut, validation (karigar required, >=1 line, qty > 0, no negative received qty).

Tests: Req 51 Tests 7, 11, 12, plus a line received across two separate receive entries.

Phase 8 — Receipt and Payment

This phase posts each receipt or payment exactly once to cash/bank and shows Balance Before, This Entry and Closing Balance from the ledger.

PHASE 8 – Receipt & Payment with Cash/Bank and balances (Req 34-39, 46).

Read the existing Cash & Bank ledger design first and explain it back to me in 5-10 lines before changing anything.

RECEIPT (money IN, usually from a customer)

- Fields: party, amount (> 0), mode/account (Cash or a specific bank account from Cash & Bank), reference (cheque/UPI no.), note. Date/time from the server.
- One transaction: receipt record -> party ledger entry -> selected cash/bank ledger entry -> audit -> outbox. Exactly ONE cash/bank ledger row per receipt. Confirm no other code path also posts it. Every ledger row references the receipt id.
- Screen and saved view: Balance Before | This Receipt | Closing Balance from the ledger (Rs 20,000 - Rs 7,000 = Rs 13,000).

PAYMENT (money OUT, usually to a supplier or karigar)

- Same structure, mirrored ledger direction. Show Balance Before | This Payment | Closing Balance.
- Warn (don't block) if the payment exceeds the amount owed or would take cash/bank negative.

Req 38 ('payment from customer -> bank account') = a Receipt with a bank account selected. The bank balance on Cash & Bank must update immediately.

Req 35 is ambiguous. Do NOT implement any automatic supplier/karigar posting from a customer receipt until I confirm what the client means. [Paste the client's answer here when you have it.]

Also: save shortcut, validation, owner edit/delete with ledger reversal, audit.

Tests: Req 51 Tests 9 and 10; bank balance after a receipt; edit and delete leave no orphan ledger rows.

Phase 9 — Reminders

This phase extends the existing Reminders module rather than building a second one.

PHASE 9 — Reminders (Req 5, 14, 24). Extend the EXISTING Reminders module; do not create a second one.

1. Fields: party, linked transaction (type + id; optional for manual), amount, reminder date, note, status (Pending / Completed / Cancelled), created_by, created_at, completed_at.
2. Manual creation from the Reminders page and from a party's page.
3. Reminders created from Sales/Purchase (Phases 5-6) link back to their bill.
4. List filters: Due today / Overdue / Upcoming / Completed / Cancelled. Actions: Mark completed, Cancel, Edit date. Overdue highlighted. Dashboard widget with due-today and overdue counts.
5. Deleting a linked bill cancels its pending reminders in the same transaction.
6. Validation: date required; a linked transaction must exist and belong to the same party.
7. Audit log for create/edit/status change; staff isolation per Phase 2.
8. (After Phase 10) optional owner notification when a reminder falls due. Note this in PROGRESS.md.

Phase 10 — Notifications and WhatsApp

This phase routes every message through an outbox written inside the accounting transaction, so nothing is sent for a failed save and nothing is lost.

PHASE 10 – Owner notifications, Receipt/Payment WhatsApp messages, 'Send Bill on WhatsApp' (Req 3, 4, 34, 37, 47).

ARCHITECTURE (separate from accounting code)

Accounting transaction -> insert notification_outbox row INSIDE the same DB transaction -> commit -> dispatcher -> NotificationService -> WhatsAppProvider.

1. notification_outbox: id, type (owner_entry_alert, receipt_message, payment_message, bill_image), entity type/id, recipient, payload JSON, status (pending, sent, failed, manual_pending, skipped), provider message id, error, attempts, created_at, sent_at.

Because the row is written inside the accounting transaction, a notification exists only if the entry committed, and none are lost.

2. Dispatcher runs after commit (Next.js after()/waitUntil, plus a Vercel Cron that retries pending/failed rows with backoff). The accounting save never waits on or fails because of WhatsApp.

3. WhatsAppProvider interface: sendText(to, text), sendImage(to, imageUrl, caption).

- ApiProvider: Meta WhatsApp Cloud API or an Indian BSP, configured via env vars (I'll supply credentials). Business-initiated messages need pre-approved templates, so build message builders that map to template parameters.

- ManualProvider (no credentials): builds a wa.me link with prefilled text. Status = manual_pending until the user confirms. NEVER marked 'sent'.

4. Owner alert on every new entry (sales, purchase, job work issue/receive, receipt, payment, orders): entry type, number, party, amount, date/time in IST, created by. The owner's WhatsApp number lives in settings.

5. Receipt/Payment message to the party after save: number, party, amount, previous balance, closing balance, date.

6. Bill image: reuse the existing bill view/print layout and render it to PNG. Include business name, bill no., date, party, item lines (qty/unit/rate/amount), GST/discount/transport/packaging/other when non-zero, total, previous balance, this bill, closing balance. Store it with a private/expiring URL.

7. 'Send Bill on WhatsApp' – one click on the saved bill:

- API provider: sends the image to the party. Button shows Sending -> Sent / Failed (Retry).

- No API, mobile: navigator.share({ files: [png] }) so the user picks WhatsApp with the image attached.

- No API, desktop: download the PNG and open wa.me/<number> with a

prefilled message, telling the user to attach the downloaded image (wa.me cannot attach files). Status = manual.

8. Delivery status badge on each entry, plus an owner-only Notifications log with Retry.

9. Full mobile numbers are used server-side only (Phase 3 masking stays intact).

Tests: provider success; provider failure (entry still saved, status failed, retry works); no credentials (manual fallback, never says 'sent'); Req 51 Test 14.

Phase 11 — Sales Orders and Purchase Orders

This phase adds orders that never touch the ledger, and lets them convert into bills that track fulfilment.

PHASE 11 – Sales Orders and Purchase Orders (Req 6, 7).

Orders are NOT bills: they never touch the ledger, stock or cash/bank.

1. Data (separate tables, or one orders table with a type if that matches existing patterns): order number in its own series (SO-0001 / PO-0001), party, order date, expected date, lines via ItemEntryGrid (item, qty, unit, rate, amount), expected amount, notes, status, created_by, created_at.

2. Statuses. Sales Order: Pending, Partially fulfilled, Completed, Cancelled. Purchase Order: Pending, Partially received, Completed, Cancelled.

3. 'Convert to Bill': opens the Sales/Purchase bill prefilled with the order's pending quantities (editable). Saving the bill links bill lines to order lines, updates fulfilled quantities, and auto-updates the status. Purchase narration defaults to 'Against PO-xxxx'.

4. Lists filterable by status and party. Also: staff isolation, owner edit/delete/cancel, audit log, owner notification via outbox, save shortcut, Alt+C / Alt+S on the forms.

5. Validation: party required, ≥ 1 line, $qty > 0$.

Tests: SO -> partial bill -> 'Partially fulfilled' -> second bill -> 'Completed'. Same flow for PO.

Phase 12 — Audit log, full testing, change report

This phase runs all 14 acceptance tests as Owner, Staff A and Staff B, and produces the final change report.

PHASE 12 – Audit log verification, acceptance testing, change report (Req 41, 50, 51, 52).

1. Audit log: verify every create/edit/delete across all entities (entries, parties, suppliers, karigars, receipts, payments, orders, reminders, staff) records user, action, entity, entity id, timestamp, and before/after for edits. The Audit Log page is owner-only, with filters for user, entity and date range. Staff cannot read audit logs.
2. Run all 14 acceptance tests in Req 51 as Owner, Staff A and Staff B (seeded test users in a non-production environment). For each test record steps, expected, actual, and pass/fail. Fix failures and re-run.
3. Also test:
 - double-press save, and a network failure mid-save (no partial data)
 - WhatsApp failure paths
 - Backspace inside vs outside inputs
 - all shortcuts in Chrome, Edge and Firefox
 - mobile layout of the grid and the bill image
 - a ledger integrity script: for every party and bank account, displayed balance must equal the ledger sum exactly
4. Typecheck, lint, build, all tests; verify a Vercel preview deployment works.
5. Write docs/CHANGE_REPORT.md covering:
 - what changed, and files/components modified
 - migrations in order, with rollback steps
 - API/server changes
 - new permissions, shortcuts and workflows
 - required env vars (e.g. WhatsApp credentials)
 - remaining limitations and open client questions
 - tests performed, with resultsBe explicit about anything not tested.

Resume prompt

Use this when a new session starts partway through the phases.

Continue the Kumkum Payal implementation. Re-read CLAUDE.md, docs/PROGRESS.md, docs/CHANGE_MAP.md and the relevant sections of docs/CLIENT_REQUIREMENTS.md. Summarise in 3-5 lines where we are and what's pending, then continue with Phase __.

Questions to settle with the client before Phase 2

Seven points in the requirements are open. Get answers before Phase 2 and paste each into the bracketed line of the phase it affects.

Question	Why it matters	Phase	Suggested default
Can staff edit or delete their own entries?	The brief only gives owner edit/delete rights explicitly.	2	Owner only
Can staff see a customer's total outstanding balance?	The old balance on a bill includes other staff members' sales.	2, 5	Show the balance figure, not the entries
What does Req 35 mean (supplier/karigar postings from a customer receipt)?	It is unclear as written and could create wrong ledger entries.	8	Don't build until clarified
Mask mobile numbers or hide them fully? Can the owner reveal them?	Affects privacy and what the API returns to staff.	3	Mask (●●●●●4321), owner can reveal
GST rates, discount before or after GST, and are transport/packaging taxed?	Changes every bill total.	5, 6	Discount first, then GST on taxable value
Will the client pay for a WhatsApp Business API account?	Fully automatic sending needs one, with approved message templates.	10	Manual fallback: one-tap share on mobile, download + prefilled chat on desktop
Labour charge: per piece, per gram or fixed? Owed when sent or when received back?	Decides how the karigar ledger moves.	7	Per line, posted on receive back